4.1 Obstacle Avoidance

(a)

In [ ]:
```python
def signed_distances(s, centers, radii):
    """Compute signed distances to circular obstacles."""
    centers = jnp.reshape(centers, (-1, 2))
    radii = jnp.reshape(radii, (-1,))

    d = jnp.linalg.norm(s[:2] - centers, axis=1) - radii

    return d
```

(b)

In [ ]:
```python
@partial(jax.jit, static_argnums=(0,))
@partial(jax.vmap, in_axes=(None, 0, 0))
def affinize(f, s, u):
    """Affinize the function `f(s, u)` around `(s, u)`."""
    A, B = jax.jacobian(f, argnums=(0, 1))(s, u)
    c = f(s, u) - A @ s - B @ u
    return A, B, c
```

(c)

Linearizing f and d around the current iterate $(s_{\cdot|t}^{(i)}, u_{\cdot|t}^{(i)})$:

$$ A_{f,k|t}^{(i)} = \frac{\partial f}{\partial s}\bigg|_{s_{k|t}^{(i)}, u_{k|t}^{(i)}}, \quad B_{f,k|t}^{(i)} = \frac{\partial f}{\partial u}\bigg|_{s_{k|t}^{(i)}, u_{k|t}^{(i)}}, \quad c_{f,k|t}^{(i)} = f(s_{k|t}^{(i)}, u_{k|t}^{(i)}) - A_{f,k|t}^{(i)} s_{k|t}^{(i)} - B_{f,k|t}^{(i)} u_{k|t}^{(i)} $$$$ A_{d,k|t}^{(i)} = \frac{\partial d}{\partial s}\bigg|_{s_{k|t}^{(i)}}, \quad c_{d,k|t}^{(i)} = d(s_{k|t}^{(i)}) - A_{d,k|t}^{(i)} s_{k|t}^{(i)} $$
The convex sub-problem is:

$$ \begin{aligned} \min_{s_{\cdot|t},\, u_{\cdot|t}} \quad & (s_{N|t} - s_\text{goal})^\top P\,(s_{N|t} - s_\text{goal}) + \sum_{k=0}^{N-1}\left[(s_{k|t} - s_\text{goal})^\top Q\,(s_{k|t} - s_\text{goal}) + u_{k|t}^\top R\,u_{k|t}\right] \\ \text{s.t.} \quad & s_{0|t} = s(t) \\ & s_{k+1|t} = A_{f,k|t}^{(i)} s_{k|t} + B_{f,k|t}^{(i)} u_{k|t} + c_{f,k|t}^{(i)}, \quad k = 0,\dots,N-1 \\ & A_{d,k|t}^{(i)} s_{k|t} + c_{d,k|t}^{(i)} \geq 0, \quad k = 0,\dots,N \end{aligned} $$

(d)

Each $d_j(s) = \|(x,y) - (x_j, y_j)\|_2 - r_j$ is convex (norm minus a constant). For any convex function g, the first-order condition gives $g(x) \geq g(\bar{x}) + \nabla g(\bar{x})^\top (x - \bar{x})$ for all x. Rearranging, $g(x) \geq \underbrace{\nabla g(\bar{x})^\top}_{A_d} x + \underbrace{g(\bar{x}) - \nabla g(\bar{x})^\top \bar{x}}_{c_d}$, so

Loading [MathJax]/extensions/Safe.js _{d,k|t}^{(i)} s_{k|t} + c_{d,k|t}^{(i)}$$

Therefore $A_{d,k|t}^{(i)} s_{k|t} + c_{d,k|t}^{(i)} \geq 0 \implies d(s_{k|t}) \geq 0$. The linearized constraint is a conservative inner approximation of the true obstacle constraint, so feasibility of the sub-problem guarantees safety.

(e)

In [ ]:
```python
def scp_iteration(f, d, s0, s_goal, s_prev, u_prev, P, Q, R):
    """Solve a single SCP sub-problem for the obstacle avoidance problem."""
    n = s_prev.shape[-1]
    m = u_prev.shape[-1]
    N = u_prev.shape[0]

    Af, Bf, cf = affinize(f, s_prev[:-1], u_prev)
    Af, Bf, cf = np.array(Af), np.array(Bf), np.array(cf)
    Ad, _, cd = affinize(
        lambda s, _: d(s), s_prev, jnp.concatenate((u_prev, u_prev[-1:]))
    )
    Ad, cd = np.array(Ad), np.array(cd)

    s_cvx = cvx.Variable((N + 1, n))
    u_cvx = cvx.Variable((N, m))

    objective = 0.0
    constraints = [s_cvx[0] == s0]

    for k in range(N):
        objective += cvx.quad_form(s_cvx[k] - s_goal, Q) + cvx.quad_form(u_cvx
        constraints += [s_cvx[k + 1] == Af[k] @ s_cvx[k] + Bf[k] @ u_cvx[k] + 
        constraints += [Ad[k] @ s_cvx[k] + cd[k] >= 0]

    objective += cvx.quad_form(s_cvx[N] - s_goal, P)
    constraints += [Ad[N] @ s_cvx[N] + cd[N] >= 0]

    prob = cvx.Problem(cvx.Minimize(objective), constraints)
    prob.solve()
    s = s_cvx.value
    u = u_cvx.value
    J = prob.objective.value
    return s, u, J
```

(f)

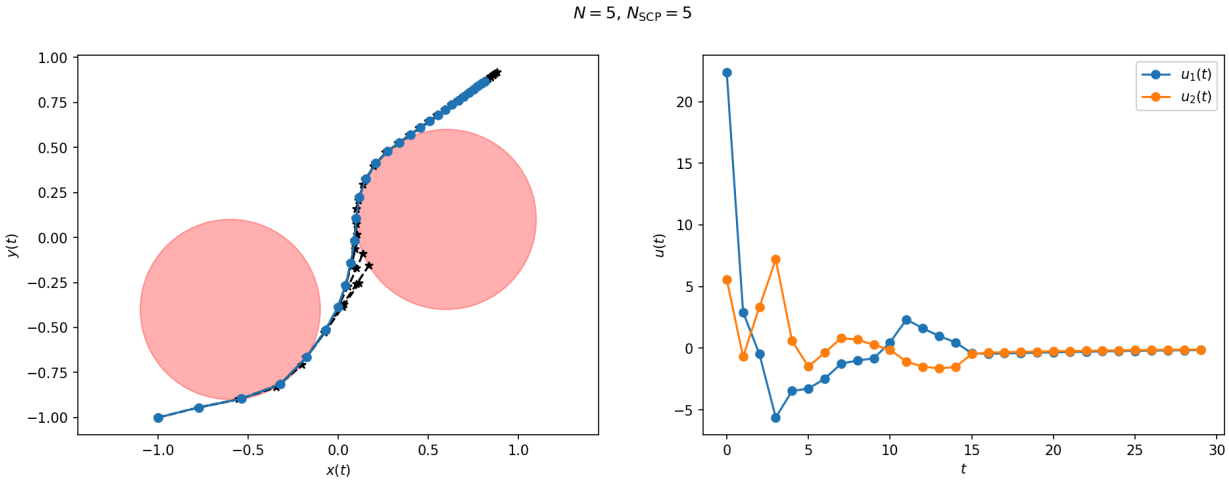
In [ ]:
```python
# Solve the MPC problem at time `t`
s_mpc[t], u_mpc[t] = solve_obstacle_avoidance_scp(
    f, d, s, s_goal, N, P, Q, R, eps, N_scp, s_init=s_init, u_init=u_init
)

# Push the state `s` forward in time with a closed-loop MPC input
s = np.array(f(s, u_mpc[t, 0]))
```

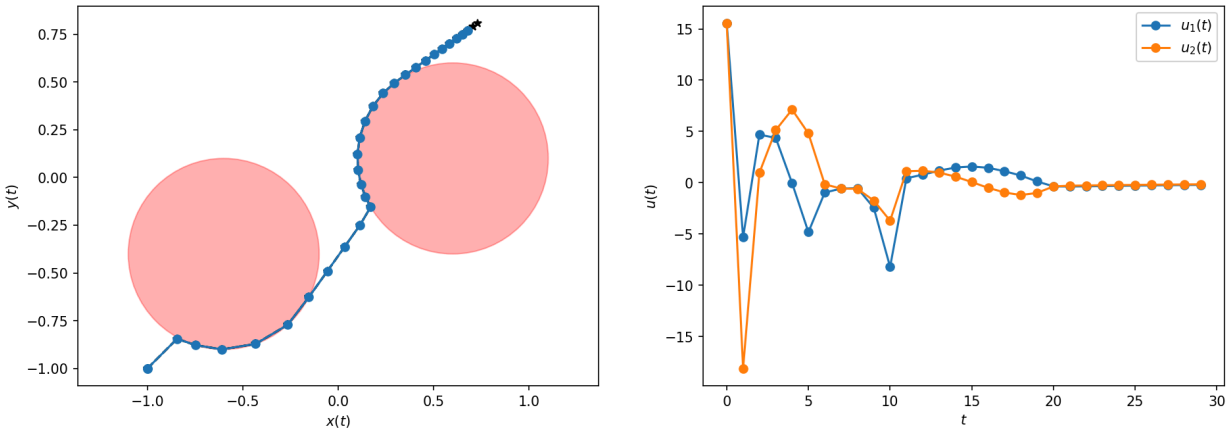
(g)

N	N_SCP	total_time (s)	total_control_cost
5	5	13.91	6.93
2	5	13.08	11.18

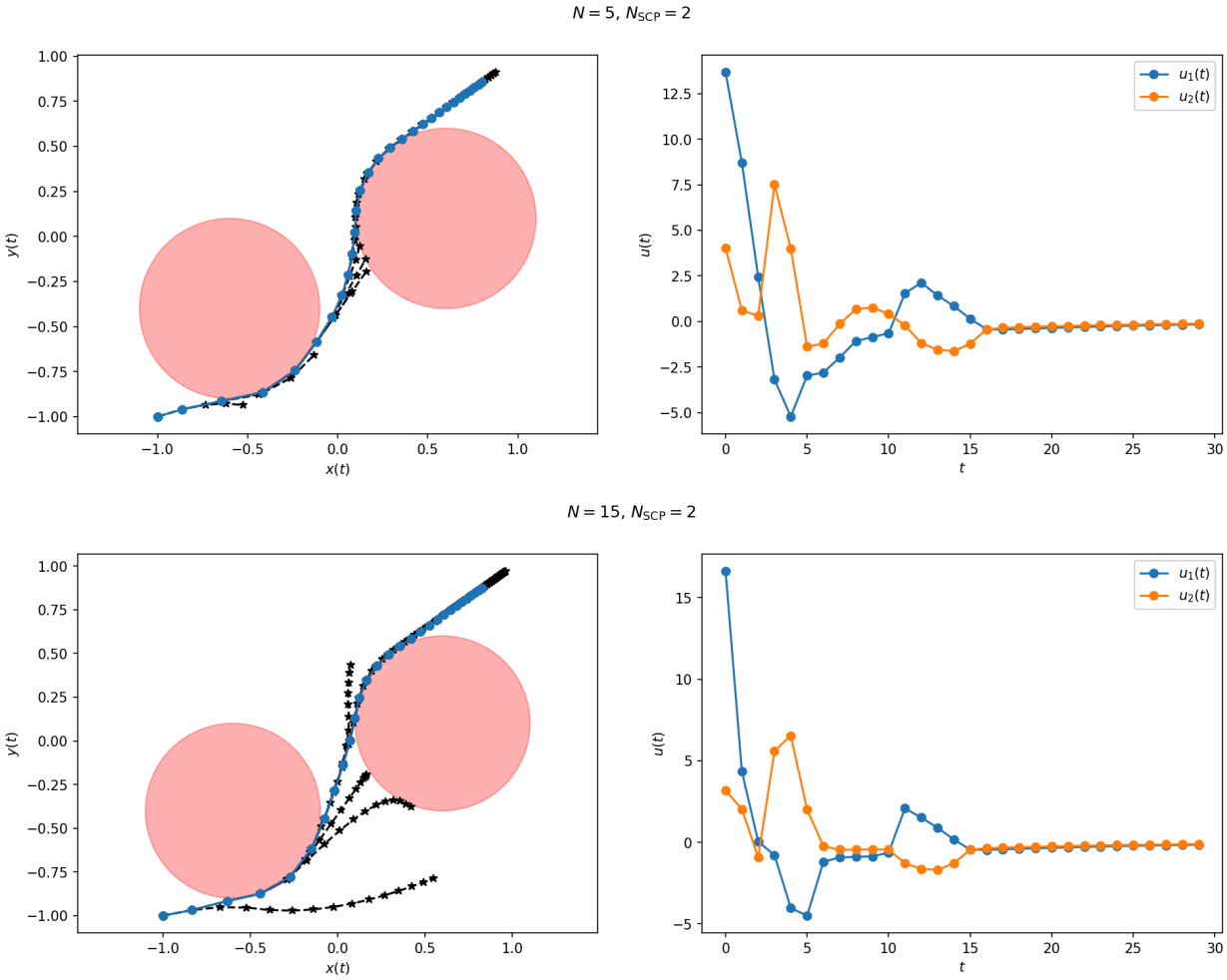
N	N_SCP	total_time (s)	total_control_cost
5	2	8.45	4.43
15	2	11.49	4.50

In [1]:
```
from IPython.display import display, Image

for fname in [
    "../soln_obstacle_avoidance_N=5_Nscp=5.png",
    "../soln_obstacle_avoidance_N=2_Nscp=5.png",
    "../soln_obstacle_avoidance_N=5_Nscp=2.png",
    "../soln_obstacle_avoidance_N=15_Nscp=2.png",
]:
    display(Image(filename=fname))
```





Loading [MathJax]/extensions/Safe.js



All four cases successfully steer the system from $(-1,-1)$ toward $(1,1)$ while avoiding both obstacles.

With $N=5$ and $N_\text{SCP}=5$, the trajectory smoothly navigates between the two obstacles, though the control inputs show a large initial spike in u_1 before settling, resulting in a cost of 6.93.

Reducing the horizon to $N=2$ (with $N_\text{SCP}=5$) makes the controller myopic, since it cannot anticipate the obstacles early enough. As a result, the trajectory hugs the obstacle boundaries more tightly and the controls become much more aggressive and oscillatory, which explains the highest control cost of 11.18.

With $N=5$ and $N_\text{SCP}=2$, the trajectory is very similar to the $N=5, N_\text{SCP}=5$ case but with smaller control magnitudes. This is the fastest run (8.45s) and achieves the lowest cost (4.43), suggesting that SCP already converges in just 2 iterations here.

Increasing the horizon to $N=15$ (with $N_\text{SCP}=2$) allows the planner to see further ahead, as shown by the longer planned trajectories. However, the actual trajectory and control cost (4.50) are comparable to the $N=5, N_\text{SCP}=2$ case, while the computation takes longer (11.49s) because of the larger optimization problem. Overall, a

moderate horizon with few SCP iterations provides the best trade-off between computation time and trajectory quality.

4.2 Widget Sales

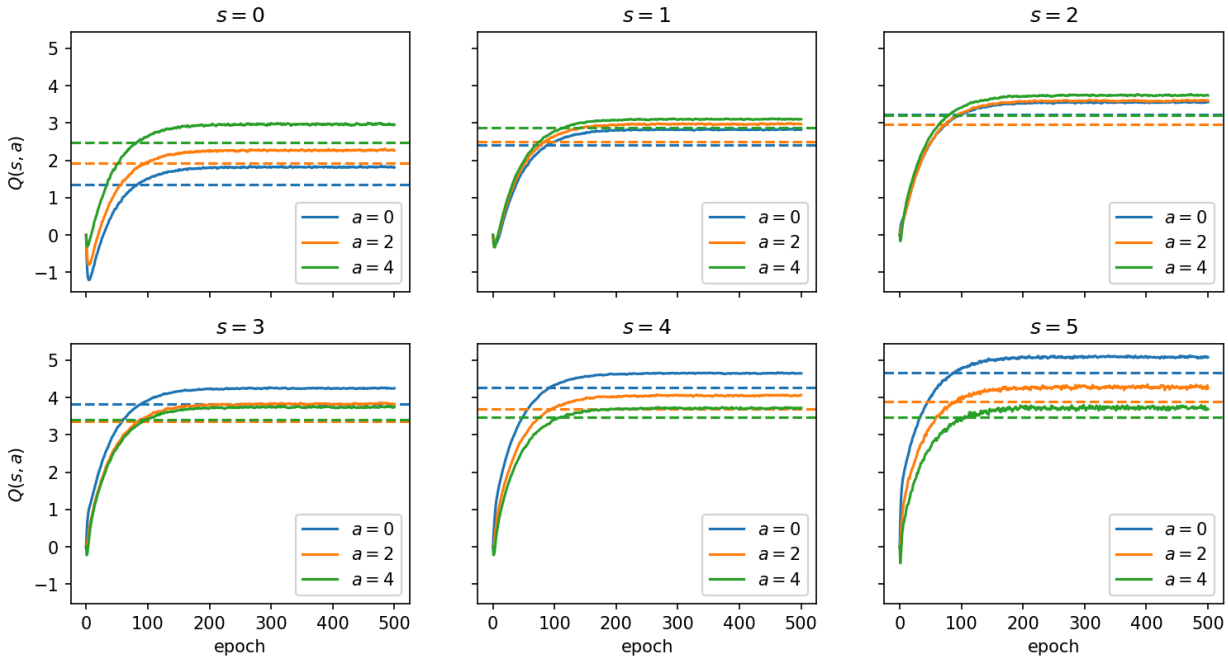
(a)

In [ ]:
```
for idx in shuffled_indices:
    s_t = int(log["s"][idx])
    a_t = int(log["a"][idx])
    r_t = log["r"][idx]
    s_next = int(log["s"][idx + 1])

    a_idx = np.searchsorted(A, a_t)
    Q[s_t, a_idx] += α * (r_t + γ * np.max(Q[s_next]) - Q[s_t, a_idx])
```

(b)

In [ ]:
```
Q_vi_new = np.zeros_like(Q_vi)
for i, s in enumerate(S):
    for j, a in enumerate(A):
        Q_vi_new[i, j] = sum(
            P[l] * (reward(s, a, d) + γ * np.max(Q_vi[transition(s, a, d)]))
            for l, d in enumerate(D)
        )
Q_vi_prev = Q_vi.copy()
Q_vi[:] = Q_vi_new
```

In [1]:
```
from IPython.display import display, Image
display(Image(filename="../widget_sales_qvalues.png"))
```



The Q-learning values (solid lines) converge toward the value iteration values (dashed lines) but do not match them exactly. For states $s=0, 1, 4, 5$ the action ranking is the same between both methods, so the optimal action does not change. However, at $s=2$ the ranking differs: Q-learning ranks $a=4 > a=2 > a=0$ while value iteration gives $a=0 > a=4$

> a=2$, leading to a different optimal action ($a=4$ vs $a=0$). At $s=3$ the best action agrees ($a=0$) but the ordering of the suboptimal actions is swapped. These differences arise because Q-learning learns from real transitions while value iteration relies on the intern's approximate model of the dynamics and demand distribution.

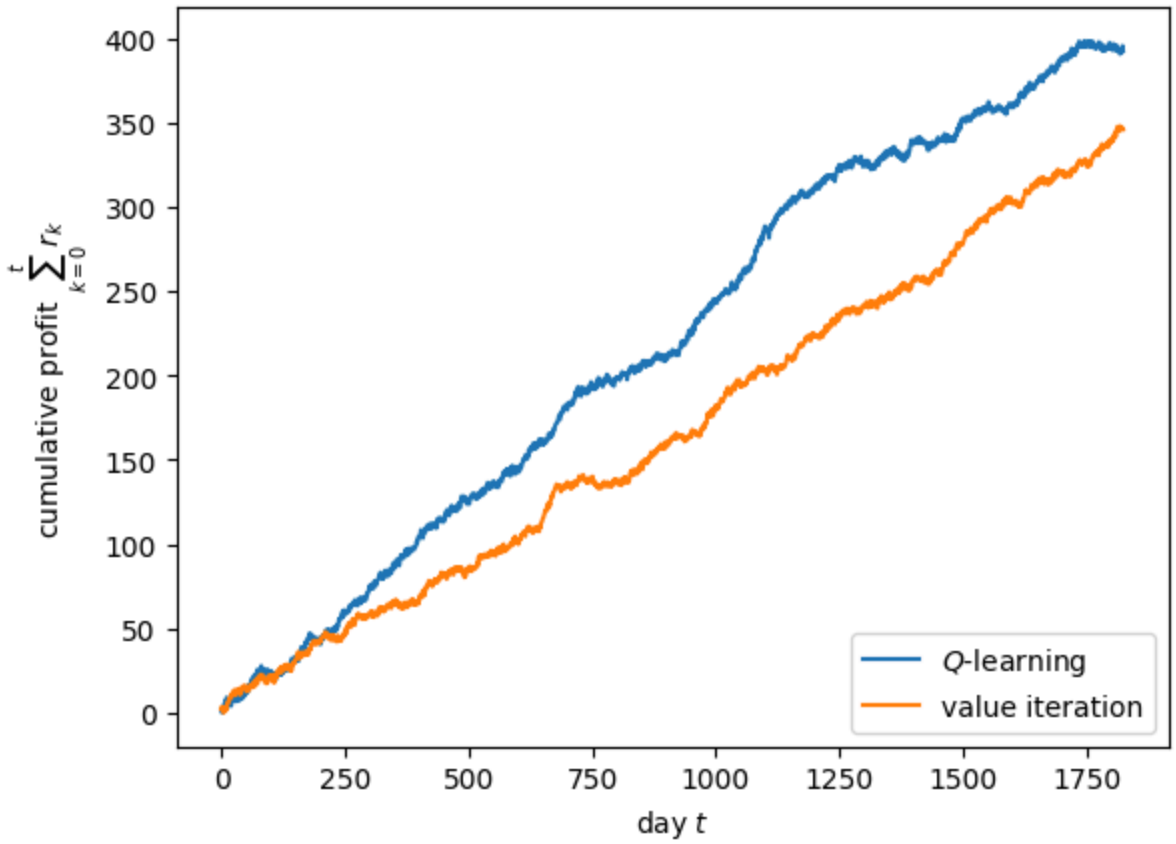
(c)

In [ ]:
```
a_opt_ql = A[np.argmax(Q, axis=1)]
a_opt_vi = A[np.argmax(Q_vi, axis=1)]

_, _, r_ql = simulate(rng, lambda s: a_opt_ql[int(s)], T)
profit_ql = np.cumsum(r_ql)

_, _, r_vi = simulate(rng, lambda s: a_opt_vi[int(s)], T)
profit_vi = np.cumsum(r_vi)
```

Optimal policy (Q-learning): $[4, 4, 4, 0, 0, 0]$

Optimal policy (value iteration): $[4, 4, 0, 0, 0, 0]$

In [2]:
```
display(Image(filename="../widget_sales_profits.png"))
```



The Q-learning policy accumulates higher profit than the value iteration policy over 5 years. The two policies differ only at $s=2$: Q-learning orders 4 widgets while value iteration orders 0. Since the intern's model is only an approximation of the true dynamics and demand distribution, value iteration is optimal with respect to that approximate model but n respect to reality. Q-learning, on the other hand, learned directly from

real historical transitions, so it may have captured the true underlying dynamics more accurately. The higher cumulative profit of Q-learning suggests that the model errors in the intern's approximation hurt the value iteration policy, particularly at $s=2$ where the two methods disagree.

4.3 CartPole

(a)

In [ ]:
```
def build_network(self) -> torch.nn.Module:
    Q=torch.nn.Sequential(
        torch.nn.Linear(self.state_dim, self.hidden_dim),
        torch.nn.ReLU(),
        torch.nn.Linear(self.hidden_dim, self.hidden_dim),
        torch.nn.ReLU(),
        torch.nn.Linear(self.hidden_dim, self.action_dim)
    )
    return Q
```

In [ ]:
```
def policy(self, state : Union[np.ndarray, torch.tensor], train : bool=False) -
    if isinstance(state, np.ndarray):
        state = torch.tensor(state).unsqueeze(0).to(self.device)
    if train:
        if random.random()<self.eps_threshold():
            return torch.randint(0, self.action_dim, (1,)).to(self.device)
        else:
            with torch.no_grad():
                return torch.argmax(self.policy_network(state)).unsqueeze(0)
    else:
        with torch.no_grad():
            return torch.argmax(self.policy_network(state)).unsqueeze(0)
```

In [ ]:
```
def train(self, env : gym.wrappers, num_episodes : int=100) -> None:
    optimizer=torch.optim.AdamW(self.policy_network.parameters(), lr=self.learn
    loss_fn=torch.nn.MSELoss()
    for episode in tqdm(range(num_episodes)):
        state, _ = env.reset()
        done = False
        while not done:
            action=self.policy(state, train=True)
            next_state, reward, terminated, truncated, _ = env.step(action.iter
            done=terminated or truncated
            sp = None if terminated else torch.tensor(next_state).unsqueeze(0)
            self.buffer.append(Transition(torch.tensor(state).unsqueeze(0).to(
            state=next_state

            if len(self.buffer)>self.batch_size:
                states, actions, targets=self.sample_buffer()
                predictions=self.policy_network(states).gather(1, actions)
                loss=loss_fn(predictions, targets)
                optimizer.zero_grad()
                loss.backward()
                torch.nn.utils.clip_grad_value_(self.policy_network.parameters
                optimizer.step()
```

DQN evaluation on CartPole-v1:

- Before training: average reward = 11.4
- After training (500 episodes): average reward = 500.0

(b)

In [ ]:
```python
def learn(self, rewards: list, log_probs: list) -> None:
    T = len(rewards)
    log_probs = torch.stack(log_probs)

    # 1) Naive REINFORCE
    # R = sum(self.gamma**t * rewards[t] for t in range(T))
    # loss = -(log_probs * R).sum()

    # 2) REINFORCE with causality trick
    # returns = torch.zeros(T)
    # for t in range(T):
    #     returns[t] = sum(self.gamma**(tp - t) * rewards[tp] for tp in range(
    # loss = -(log_probs * returns).sum()

    # 3) REINFORCE with causality trick and baseline to "center" the returns
    returns = torch.zeros(T)
    for t in range(T):
        returns[t] = sum(self.gamma**(tp - t) * rewards[tp] for tp in range(t,
    baseline = sum(self.gamma**t * rewards[t] for t in range(T)) / T
    loss = -(log_probs * (returns - baseline)).sum()
```

In [ ]:
```python
from IPython.display import display, Image

print("Naive REINFORCE:")
display(Image(filename="../reward_curve_naive.png"))
print("REINFORCE with causality trick:")
display(Image(filename="../reward_curve_causality.png"))
print("REINFORCE with causality trick + baseline:")
display(Image(filename="../reward_curve_baseline.png"))
```

DQN is more sample efficient than REINFORCE because it stores transitions in a replay buffer and performs a gradient step at every environment step, reusing past experience multiple times. REINFORCE, on the other hand, collects trajectories online with the current policy and discards them after a single gradient update, making it inherently less data efficient. As a result, DQN reaches an average reward of 500.0 after 500 episodes while all three REINFORCE variants achieve lower rewards.

Among the REINFORCE variants, the causality trick improves learning since the variance from past rewards is removed from the gradient computation, allowing each action to be reinforced only based on its future consequences. The baseline variant further removes state-relative variance for a similar final performance as the causality trick.

In [ ]:

4.4 Learning LQR

(a)

Model-based RLS assumes linear dynamics $x_{t+1} = Ax_t + Bu_t$ and quadratic cost $c(x,u) = x^\top Q x + u^\top R u$. It explicitly learns estimates $\hat{A}, \hat{B}, \hat{Q}, \hat{R}$ from data via recursive least-squares, then solves the Riccati equation to obtain the optimal gain K. This method exploits the most problem structure and converges in roughly 10 episodes, as seen in Figure 1.

Model-free policy iteration assumes the Q-function has a quadratic structure $Q_K(x,u) = (x,u)^\top H_K (x,u)$, but does not learn a dynamics model. It fits H_K directly from observed temporal-difference errors using least-squares, then updates the policy as $K \leftarrow -H_{K,22}^{-1} H_{K,12}^\top$. Since it avoids estimating A, B and only relies on the quadratic Q-function assumption, it uses less structural information than model-based RLS and converges in roughly 1000 episodes.

Model-free policy gradients makes no structural assumptions about the dynamics or the cost. It treats the policy as a parametric black box $\pi_K(u \mid x) = \mathcal{N}(Kx, \Sigma)$ and updates K via Monte Carlo REINFORCE gradients. Because it does not exploit any problem structure, it requires roughly 10,000 episodes and still has not fully converged.

Overall, the more structural assumptions a method incorporates, the faster and more precisely it converges. Model-based RLS reaches the smallest gain error $\|K^{(i)} - K^\star\|_F$ and the lowest cost, while policy gradients retains a significant residual error even after 10,000 episodes. In this case, the true underlying system is indeed linear with quadratic cost, so exploiting these assumptions does not introduce any model mismatch or bias. Had the true system been nonlinear or the cost non-quadratic, the stronger assumptions of model-based RLS and policy iteration would have introduced a bias, potentially making the assumption-free policy gradient method more robust despite its slower convergence.

(b)

Model-based RLS can most naturally incorporate prior beliefs on A, B, Q, R through regularization in the least-squares fits, which corresponds to Bayesian regression with a prior. For instance, a Gaussian prior on A, B translates directly into a ridge penalty centered at the prior mean, biasing estimates toward the prior especially when data is scarce.

Model-free policy iteration does not estimate A, B directly, so priors on the dynamics cannot be incorporated in a straightforward way. However, prior beliefs on the cost matrices

Q, R could inform the initialization or regularization of the H_K fit, since H_K implicitly depends on Q and R.

Model-free policy gradients has no internal model to regularize, making it the hardest method in which to incorporate prior beliefs. At best, prior knowledge of A, B could be used to compute a good initial policy $K^{(0)}$ (e.g., via Riccati on the prior mean), and prior knowledge of cost magnitudes could help set the learning rate or standardization parameters.

4.5 Lunar Lander

(a)

The code computes the advantage estimate as:

```
advantages = returns - values
```

where `returns` are the Monte Carlo tail returns $G_t = \sum_{\tau=t}^{T} \gamma^{\tau-t} r_\tau$ computed from a single episode rollout via `compute_returns`, and `values` = $V_w^\pi(x_t)$ are the critic's learned estimates from the `value_head` of the shared network. This corresponds to the advantage estimator:

$$\delta^\pi = G_t - V_w^\pi(x_t)$$

On the bias-variance spectrum, this estimator is **low bias, high variance**. It is unbiased since G_t is a Monte Carlo estimate of $Q^\pi(x_t, u_t)$, so the advantage estimate is an unbiased estimate of $A^\pi(x_t, u_t)$. However, it has high variance since it relies on the full trajectory return, which accumulates randomness across all timesteps. Subtracting the baseline $V_w^\pi(x_t)$ reduces variance without introducing bias, because the baseline depends only on the state and not on the action, so it vanishes in expectation when computing the policy gradient.

(b)

Returns can vary by orders of magnitude across training, as early episodes may yield very negative returns while later ones are large and positive. Without standardization, gradient magnitudes would be inconsistent across episodes, making learning unstable. The code standardizes returns using a running EMA of the mean and variance:

```
standardized_returns = (returns - return_ema) / (np.sqrt(return_emv)
+ 1e-6)
```

This keeps the critic targets and actor gradient weights at a consistent scale throughout training, similar in spirit to batch normalization, and improves learning stability.

(c)

`jax.lax.stop_gradient` prevents gradients from flowing through the advantage estimate when computing the actor loss:

```
actor_loss = jnp.sum(-action_log_probs *
jax.lax.stop_gradient(advantages) * mask)
```

Without it, the gradient of `actor_loss` would flow through `advantages = returns - values`, causing the actor update to also push the critic's `values` toward `returns`, effectively mixing actor and critic gradients through the shared trunk. This is incorrect: the policy gradient theorem requires the advantage to be treated as a fixed scalar weight δ^π when differentiating with respect to θ. The critic already learns

separately through `critic_loss = sum(advantages^2)`, so `stop_gradient` ensures the two losses remain decoupled as intended.

(d)

An alternative approach would be model-based trajectory optimization, such as iLQR or SCP (as in Problem 1). If the lunar lander dynamics were known or could be identified from data, one could linearize the dynamics at each timestep and solve a finite-horizon optimal control problem to compute a control sequence. Compared to the model-free A2C approach, trajectory optimization is significantly more sample efficient and can provide stronger guarantees on the quality of the solution. However, it requires an accurate dynamics model, since lunar lander contact forces and thruster dynamics are difficult to model precisely and model mismatch would introduce suboptimality or instability. The model-free RL approach here makes no assumptions on the dynamics and is more general, at the cost of requiring thousands of environment interactions to converge.